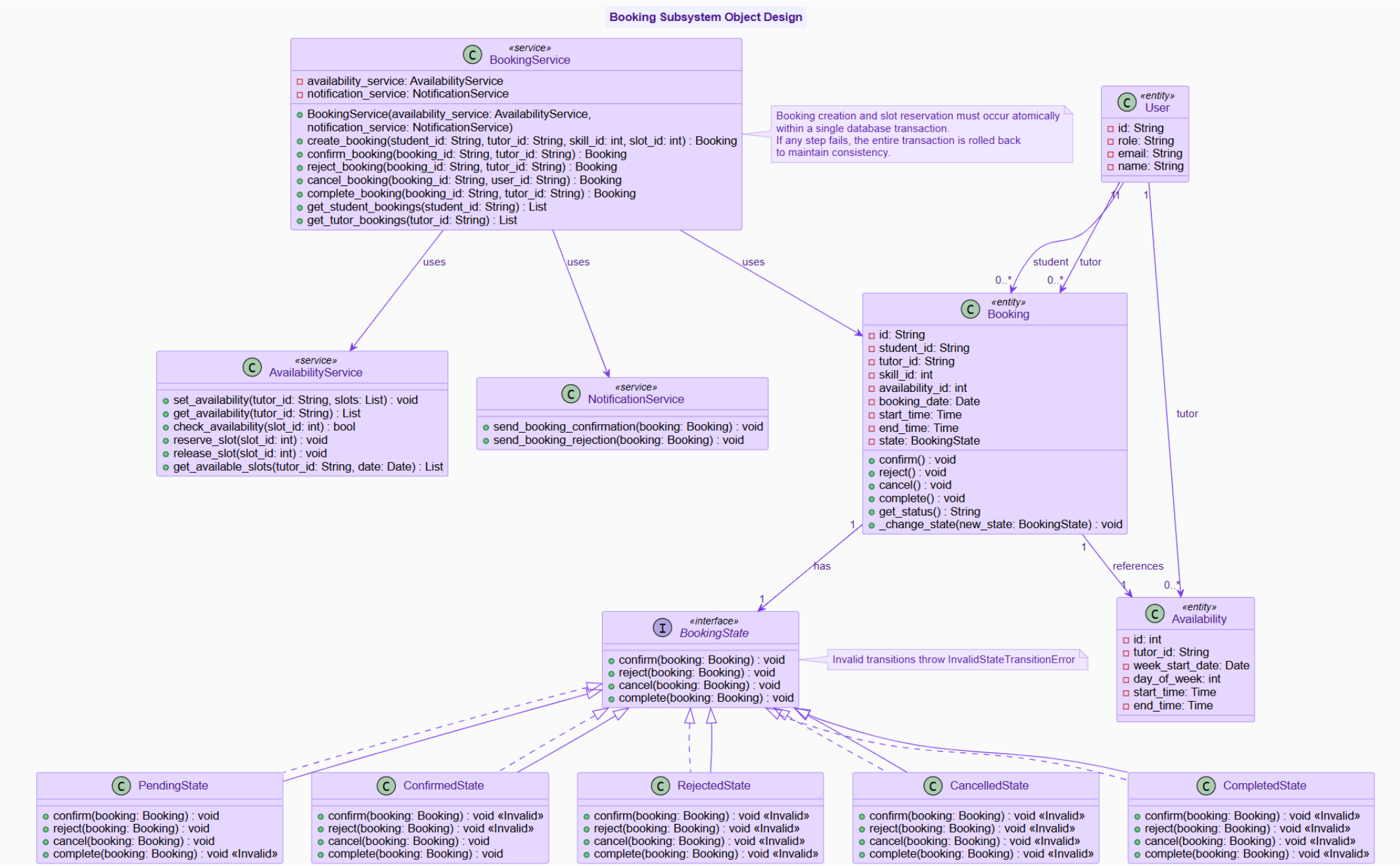
Object Design: Booking Service Class Diagram

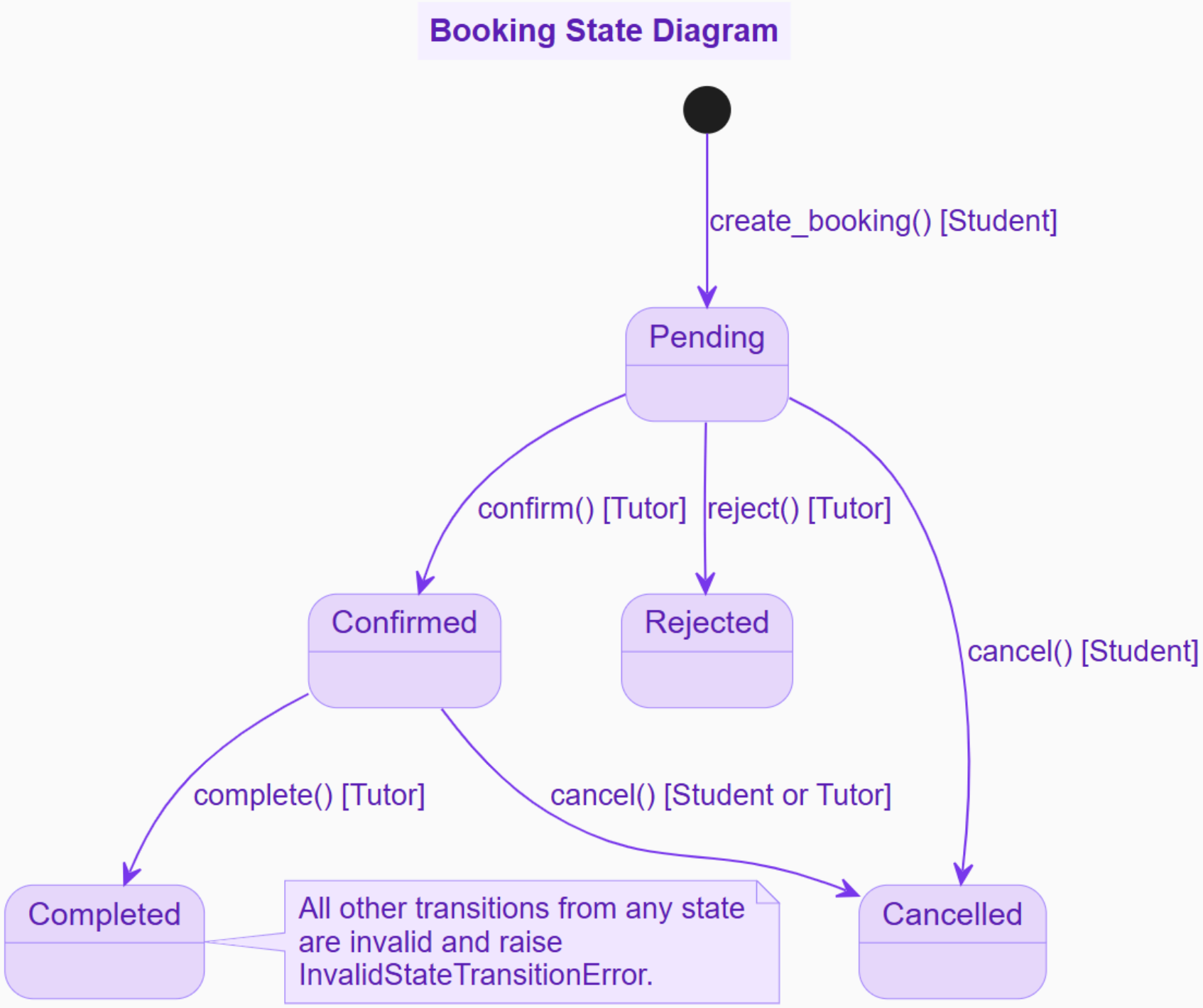


Design Patterns Used :

Pattern	Problem	Solution	In the Diagram	Role
Service Layer	Business logic mixed with HTTP	Extract logic to separate service classes	AvailabilityService, BookingService	Organizes business logic into classes
State Pattern	Complex if-else for booking status	Each state as its own class	BookingState + 5 concrete states	Manages booking lifecycle inside those classes
Dependency Injection	Hard-coded dependencies	Pass dependencies from outside	BookingService depends on AvailabilityService	Connects services together cleanly

State Transition Table :

CURRENT STATE	Confirm	Reject	Cancel	Complete
Pending	confirmed	rejected	cancelled	not allowed
Confirmed	not allowed	not allowed	cancelled	completed
Rejected	not allowed	not allowed	not allowed	not allowed
Cancelled	not allowed	not allowed	not allowed	not allowed
Completed	not allowed	not allowed	not allowed	not allowed


Booking State Diagram
create_booking() [Student]
Pending
confirm() [Tutor]
reject() [Tutor]
cancel() [Student]
Confirmed
Rejected
complete() [Tutor]
cancel() [Student or Tutor]
Completed
All other transitions from any state are invalid and raise InvalidStateTransitionError.
Cancelled